API Gateway — Mimari Analiz Raporu

> Tarih: 2026-04-30

Genel Skor

Alan	Puan	Yorum
Mimari	7/10	İyi yapılandırılmış, eksik pattern'lar var
Kod Kalitesi	6/10	Okunabilir ama error handling zayıf
Güvenlik	6/10	JWT/CB iyi, config güvenliği zayıf
Test Coverage	3/10	Sadece circuit breaker test var
Observability	6/10	Prometheus var, tracing yok

Kritik Sorunlar

1. TLS Doğrulaması Kapalı

Dosya: `internal/handler/websocket.go:26`

```
TLSClientConfig: &tls.Config{InsecureSkipVerify: true}
```

Şu an: Sorun değil. Ngrok zaten şifrelemeyi kendi hallediyor, lokalden dışarı çıkmıyor.

Sunucu alınınca yapılacaklar:

Mimari şu şekilde kurulacak:

```
Kullanıcı
    ↓ HTTPS
  Nginx  ← Let's Encrypt sertifikası burada tanımlanır
    ↓ HTTP (iç network)
api-gateway
    ↓ HTTP
market-data | exchange | auth | frontend
```

- **Nginx** kurulup domain'e bağlanır, TLS'i Nginx halleder
- **Cloudflare** ile ücretsiz sertifika alınır — aynı zamanda DDoS koruması ve CDN de dahil, DNS Cloudflare'e taşınır
- **Kod tarafında:** `InsecureSkipVerify: true` satırı silinir, servisler artık HTTP iç network üzerinden konuştuğu için TLS config'e gerek kalmaz

---

2. JWT Fallback Secret

Dosya: `internal/middleware/auth.go:23`

```
secret = "default-secret-change-in-production"
```

Şu an (local): docker-compose'da `JWT_SECRET=supersecretkey123` set edilmiş, fallback devreye girmiyor. Sorun yok.

Prod'da yapılması gereken:

Fallback satırı tamamen kaldırılmalı, secret yoksa uygulama başlamasın. Bunu yapmanın faydası: yanlış config ile prod'a çıkma ihtimali sıfırlanır, uygulama başlamayı reddeder ve log'a net hata yazar — sessizce güvenlik açığıyla çalışmak yerine.

```go
// config/config.go — startup'ta zorunlu kıl
secret := os.Getenv("JWT_SECRET")
if secret == "" {
    log.Fatal("JWT_SECRET is required")
}
```

Sunucuda `.env` dosyasına güçlü bir secret ekle:

```
JWT_SECRET=$(openssl rand -hex 32)
```

Bu komutu bir kere çalıştır, çıkan değeri `.env`'e yaz, bir daha dokunmuyorsun.

Token geçersizse ne olur:

- **api-gateway:** 401 döner, detay vermez
- **Frontend:** 401 alınca kullanıcıyı login sayfasına yönlendirir, tekrar login olunca yeni token alınır

---

3. Redis Hata Durumunda Blacklist Bypass

Dosya: `internal/middleware/auth.go:70-75`

Logout olan kullanıcının token'ı Redis blacklist'ine ekleniyor. Redis down olursa bu kontrol atlanıyor — logout olmuş kullanıcı sisteme girebiliyor.

Şu an (local): Redis docker-compose'da çalıştığı için pratikte sorun yok.

Prod'da: Redis'in anlık restart veya memory sorunu olabilir. Çözüm: Redis hata verirse de isteği reddet (fail closed).

```go
if err != nil {
    http.Error(w, `{"error":"unauthorized"}`, 401)
    return
}
```

---

4. WebSocket CheckOrigin Her Zaman `true`

Dosya: `internal/handler/websocket.go:18`

```go
CheckOrigin: func(r *http.Request) bool { return true }
```

Şu an (local): Sadece sen kullandığın için sorun yok.

Prod'da yapılması gereken: `ALLOWED_ORIGIN` env var zaten config'de mevcut, tek satır değişiklikle her katman kendi işini yapar (defense in depth — JWT yetkilendirme yapar, CheckOrigin origin kontrol eder, biri devre dışı kalırsa diğeri tutar):

```go
CheckOrigin: func(r *http.Request) bool {
    return r.Header.Get("Origin") == allowedOrigin
}
```

Sunucudaki `.env` dosyasına domain'i yaz:

```
ALLOWED_ORIGIN=https://your-domain.com
```

---

5. `/metrics` ve `/swagger` Endpoint'leri Auth Yok

Dosya: `cmd/main.go:75-81`

- `/metrics` → sistemin iç durumu (istek sayısı, circuit breaker, rate limit istatistikleri) herkese açık
- `/swagger` → tüm API endpoint listesi herkese açık

Şu an (local): Teknik olarak herkes erişebilir ama local'de dışarıya açık olmadığı için pratikte sorun yok.

Prod'da yapılması gereken: Nginx kurulurken bu iki endpoint'e IP kısıtlaması eklenir — kod değişikliği yok, Nginx config'de hallolur:

```nginx
location /metrics {
    allow <sunucu-ip>;
    deny all;
}
```

```
location /swagger {
    allow <sunucu-ip>;
    deny all;
}
```

---

6. Graceful Shutdown Yok

Dosya: `cmd/main.go:174`

Şu an sunucu kapatılınca (deploy, restart) o anda işlenmekte olan tüm istekler anında kesilir. Exchange, order, pozisyon gibi kritik işlemler yarıda kalabilir.

Şu an (local): Restart nadiren yapılır, yarım kalan istek test ortamında sorun olmaz.

Prod öncesi yapılması gereken: Her deploy'da aktif kullanıcıların istekleri kesilebilir — özellikle para içeren işlemlerde ciddi sorun. Kod değişikliği küçük, prod öncesi yapılmalı:

```go
// şu an:
log.Fatal(http.ListenAndServe(":"+cfg.Port, r))

// olması gereken:
srv := &http.Server{Addr: ":" + cfg.Port, Handler: r}
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt,
syscall.SIGTERM)
defer stop()
go srv.ListenAndServe()
<-ctx.Done()
srv.Shutdown(context.Background())
```

Sinyal gelince yeni istek almayı durdurur, mevcut isteklerin bitmesini bekler, sonra kapanır.

---

7. Request Body Size Limit Yok

Sınırsız body kabul ediliyor — biri kasıtlı büyük istek gönderirse sunucu belleği dolar, çöker.

Şu an (local): Sadece geliştirici kullandığı için sorun yok.

Şimdiden yapılması gereken: Projede dosya/resim yükleme yok, tüm endpointler JSON alıyor. En büyük istek bile 50KB'ı geçmiyor — global 1MB limit fazlasıyla yeterli. Endpoint bazlı küçük limitler (auth: 4KB) auth-service'in kendi sorumluluğu.

```go
// cmd/main.go — global middleware'lere ekle
r.Use(func(next http.Handler) http.Handler {
    return http.MaxBytesHandler(next, 1<<20) // 1 MB
})
```

Prod'da ekstra yapılacak bir şey yok, kod değişikliği yeterli.

Orta Öncelikli Sorunlar

8. Config'de Hardcoded Ngrok URL'leri

Dosya: `config/config.go:24-30`

Şu an (local): Ngrok fallback olarak config.go'ya hardcoded — env var set edilmezse bu URL'ler devreye giriyor. Ngrok URL'leri birkaç saatte expire olabiliyor ve kaynak koda commit edilmiş durumda.

Prod'da yapılması gereken: Tüm servisler aynı sunucuda docker-compose ile çalışacağı için ngrok tamamen kalkıyor. `.env` dosyasına gerçek container adreslerini yaz, config.go'daki ngrok URL'lerini sil.

---

9. Health Handler Her Çağrıda Yeni `http.Client` Açıyor

Dosya: `internal/handler/health.go:78`

Şu an her `/health` çağrısında yeni bir `http.Client` oluşturuluyor — her seferinde yeni TCP bağlantısı açılıp kapanıyor. TCP bağlantısı açmak ~50-100ms maliyet, her seferinde bu ödeniyor.

Şu an (local): `/health` nadiren çağrıldığı için fark edilmez.

Prod'da: Monitoring sistemi `/health`'i sürekli çağırır — gereksiz TCP bağlantısı açılıp kapanmaya devam eder.

Çözüm: Dosya seviyesinde singleton client tanımla, tüm health check'ler bunu paylaşır:

```
// handler/health.go — dosyanın üstüne ekle
var healthClient = &http.Client{Timeout: 3 * time.Second}

// checkService içinde yeni client yerine bunu kullan
res, err := healthClient.Do(req)
```

---

10. Rate Limiter Fallback Bellek Sızıntısı

Dosya: `internal/middleware/ratelimit.go:74`

Redis down olunca her unique IP için in-memory limiter oluşturuluyor, hiçbiri temizlenmiyor.

Şu an (local): Redis hep ayakta olduğu için fallback neredeyse hiç devreye girmiyor. Sorun yok.

Prod'da: Redis'in anlık sorunu olduğunda her unique IP bellekte birikir, Redis düzelince de temizlenmiyor — uzun süreli Redis sorunu yaşanırsa bellek şişebilir. Redis prod'da stabil çalıştığı sürece nadiren tetiklenir, düşük öncelikli.

Çözüm: Fallback map'e TTL ekle — belirli süre kullanılmayan IP'leri periyodik olarak temizle. Ya da Redis düzelince map'i komple sıfırla.

---

11. Public WebSocket'lere Hiçbir Koruma Yok

Dosya: `cmd/main.go:136-139`

Public WS endpoint'lerinde üç koruma da eksik — kodun kendi yorumunda bile yazıyor:

```
// WebSocket — circuit breaker yok, rate limit yok, timeout yok
r.Handle("/ws", handler.NewWebSocketProxy(cfg.MarketDataURL))
r.Handle("/ws/quotes/*", handler.NewWebSocketProxy(cfg.MarketDataURL))
r.Handle("/ws/orderbook", handler.NewWebSocketProxy(cfg.MarketDataURL))
```

- **Circuit breaker yok** → market-data çökünce HTTP istekleri kesilir ama WS bağlantıları o servise gitmeye devam eder, çöken servisi daha da zorlar
- **Rate limit yok** → biri bot ile sınırsız WS bağlantısı açabilir, sunucu kaynakları tükenir
- **Timeout yok** → bağlantı sonsuza kadar açık kalabilir

Şu an (local): Sadece sen kullanıyorsun, sorun yok.

Prod öncesi yapılması gereken: Sadece `cmd/main.go` değişiyor:

```
r.With(rlA.Middleware).Handle("/ws",
marketDataCB.Wrap(handler.NewWebSocketProxy(cfg.MarketDataURL)))
r.With(rlA.Middleware).Handle("/ws/quotes/*",
marketDataCB.Wrap(handler.NewWebSocketProxy(cfg.MarketDataURL)))
r.With(rlA.Middleware).Handle("/ws/orderbook",
marketDataCB.Wrap(handler.NewWebSocketProxy(cfg.MarketDataURL)))
```

---

12. X-Forwarded-For Spoofing

Dosya: `internal/middleware/ratelimit.go:104`

Rate limiter IP'ye göre kısıtlama yapıyor ama IP'yi kullanıcının yazabildiği `X-Forwarded-For` header'ından alıyor. Biri her istekte farklı IP yazarsa rate limit hiç tetiklenmiyor — sınırsız istek atabilir.

Şu an (local): Sadece sen kullanıyorsun, sorun yok.

Prod'da: Cloudflare kullanılacağı için bu sorun büyük ölçüde çözülüyor. Cloudflare gerçek IP'yi `CF-Connecting-IP` header'ında gönderiyor, bunu spoof etmek mümkün değil. Kod değişikliği gerekiyor:

```
ip := r.Header.Get("CF-Connecting-IP")
if ip == "" {
    ip, _, _ = net.SplitHostPort(r.RemoteAddr)
}
```

---

13. `/api/v1/auth/*` Wildcard Rate Limitsiz

Dosya: `cmd/main.go:95`

Login ve register'a ayrı rate limit var ama diğer auth endpoint'leri (`/api/v1/auth/*` wildcard'ı) korumasız:

```
r.With(rlAuthLogin.Middleware).Handle("/api/v1/auth/login", ...)      // rate
limit var
r.With(rlAuthRegister.Middleware).Handle("/api/v1/auth/register", ...) // rate
limit var
r.Handle("/api/v1/auth/*", ...)                                        // rate
limit yok!
```

Şu an (local): Sadece sen kullanıyorsun, sorun yok.

Prod'da: Logout, refresh-token gibi endpoint'lere sınırsız istek atılabilir — auth servisi yükü artar. Çözüm basit, prod öncesi yapılmalı:

```
r.With(rlAuthLogin.Middleware).Handle("/api/v1/auth/*", authCB.Wrap(...))
```

---

14. `NewProxy()` Her Route'da Ayrı Instance Oluşturuluyor

Dosya: `cmd/main.go:102-134`

Aynı upstream'e giden 20+ route var ama her biri için ayrı `NewProxy()` çağrılıyor — yani 20 ayrı bağlantı havuzu oluşuyor. `httputil.ReverseProxy` thread-safe tasarlanmış, tek instance tüm route'lar tarafından paylaşılabilir.

Şu an: Çalışıyor ama bağlantılar verimli kullanılmıyor, config değişince 20 yerde değiştirmek gerekiyor.

Olması gereken: Tek instance oluştur, hepsi paylaşsın:

```
marketDataProxy := handler.NewProxy(cfg.MarketDataURL)
exchangeProxy   := handler.NewProxy(cfg.ExchangeURL)

r.Handle("/api/v1/quotes/*",  marketDataCB.Wrap(marketDataProxy))
r.Handle("/api/v1/history/*", marketDataCB.Wrap(marketDataProxy))
// ...
```

---

15. Çift Loglama — İki Logger Aynı Anda Aktif

Dosya: `cmd/main.go:69-73`

İki logger aynı anda aktif, her istek iki kez loglanıyor:

- `middleware.Logger` (chi built-in) → method, path, status, latency

- `apimiddleware.RequestLogger` (bizim yazdığımız) → method, path, status, latency + user ID + request ID + IP

Bizim logger zaten chi'ninkinin yaptığı her şeyi yapıyor, üstüne 3 ekstra bilgi daha yazıyor. Chi'ninkine gerek yok.

Çözüm: `cmd/main.go`'dan chi'nin logger'ını kaldır:

```
// kaldır:
r.Use(middleware.Logger)

// bırak:
r.Use(apimiddleware.RequestLogger)
```

---

16. `getEnvInt` / `getEnvDuration` İki Yerde Tanımlı

Dosya: `cmd/main.go:19` ve `middleware/circuitbreaker.go:39`

Aynı fonksiyon iki dosyada birebir kopyalanmış. Yarın bu fonksiyona bir şey eklemek gerekirse iki yerde değiştirmek gerekiyor — biri unutulursa ikisi farklı davranmaya başlar.

Çözüm: İkisi de kaldırılır, yerine `internal/env/env.go` oluşturulur:

```
// internal/env/env.go
package env

func GetInt(key string, def int) int { ... }
func GetDuration(key string, def time.Duration) time.Duration { ... }
```

Her iki dosya da oradan import eder, değişiklik gerekince tek yerden hallolur.

---

17. Circuit Breaker Half-Open'da Thundering Herd

Dosya: `internal/middleware/circuitbreaker.go:86-98`

Şu an (local): Aynı anda çok fazla kullanıcı yok, fark etmez.

Prod'da: Servis çöktüğünde circuit breaker Open'a geçiyor. 30 saniye boyunca biriken tüm istekler Half-Open'a geçince aynı anda upstream'e gidiyor — yeni kalkan servisi tekrar çökertebilir, döngüye girer.

Prod öncesi yapılması gereken: Half-Open'da sadece 1 istek geçmeli, sonuca göre karar verilmeli — başardıysa diğerlerini geçir, başaramadıysa tekrar Open'a dön. Geri kalan istekler 503 alır.

---

18. `/health` Endpoint'inde Cache Yok — Load Multiplier

Dosya: `internal/handler/health.go:26`

Şu an: Her `/health` çağrısı anında market-data ve exchange servislerine birer HTTP isteği gönderiyor. Monitoring sistemi 10 saniyede bir çağırsa sorun yok, ama kullanıcı tarafından çağrılıyorsa 100 istek = 200 outgoing request.

Çözüm: Sonuçları 5-10 saniye in-memory cache'e al, aynı süre içindeki isteklere cache'den dön.

Küçük Ama Önemli

#	Sorun	Dosya	Ne Zaman
19	`log.Printf` yerine JSON logging olmalı, prod'da log analizi zorlaşıyor	`middleware/logging.go`	Prod öncesi

---

Eklenebilecek Yeni Özellikler

Özellik	Etki	Zorluk
Distributed tracing (OpenTelemetry)	Yüksek	Orta
Response caching (Redis)	Orta	Orta
Structured JSON logging (slog/zerolog)	Orta	Kolay
Config validation at startup	Orta	Kolay
Request deduplication	Düşük	Zor
Schema validation middleware	Düşük	Zor

Açıklamalar

Distributed tracing (OpenTelemetry) Bir istek gateway'e girip auth → exchange servisine gidince "nerede ne kadar sürdü" şu an görülemiyor. OpenTelemetry her isteğe trace ID ekler, tüm servislerdeki süreyi Grafana'da görsel olarak izlemek mümkün olur. Prod'da yavaş istek şikayeti gelince nerede takıldığını anında bulursun.

Response caching (Redis) `/api/v1/quotes/BTC` gibi sık çağrılan endpointlerin cevabını Redis'te 2-3 saniye saklayarak upstream'e giden istek sayısını düşürür. 100 kullanıcı aynı anda BTC fiyatı isterse market-data servisine 1 istek gider. Market-data servisini aşırı yükten korur.

Structured JSON logging (slog) Şu an `log.Printf` ile düz metin log üretiliyor. `slog` ile JSON çıktı üretilirse Datadog, Grafana, ELK gibi araçlar logu otomatik parse edebilir. Go 1.21'de built-in gelir, ekstra paket gerekmez. `cmd/main.go`'da tek satır ile aktif edilir. Ayrıca review'de 20. madde olarak da yazılı.

Config validation at startup `JWT_SECRET` boş veya `MARKET_DATA_URL` yanlış yazılmış olsa uygulama şu an sessizce başlar, hata ilk istek gelince çıkar. Başlangıçta tüm zorunlu env değişkenleri kontrol edilirse uygulama yanlış config ile hiç ayağa kalkmaz — hatayı erkenden yakalar.

Request deduplication Ağ kopması olunca frontend aynı order isteğini 2 kez gönderebilir, bu durumda çift emir açılabilir. Her isteğe benzersiz bir key (user_id + endpoint + zaman penceresi) atanıp Redis'te kısa süre saklanarak tekrar eden istekler engellenir. Şu an proje için erken, order hacmi arttıkça değerlendirilebilir.

Schema validation middleware Gelen request body'nin beklenen formatta olup olmadığını (zorunlu alanlar, tip kontrolü) upstream servise ulaşmadan gateway'de kontrol etmek. Şu an exchange ve auth servisleri kendi validasyonunu yapıyor, gateway'de tekrar yapmak overengineering olur. İleride public API açılırsa değerlendirilebilir.